段错误的根因：为什么解引用空指针通常会崩

嵌入式 Linux 手记　2026-10-27　公众号文章 · 原链接 mp.weixin.qq.com

先猜输出。三段代码在 Linux x86-64 上各是什么下场?

```
int *p = NULL;
int a = *p;           /* 甲：空指针解引用 */

int *q;               /* 未初始化，栈上垃圾值 */
int b = *q;           /* 乙：野指针解引用 */

char buf[8];
buf[1000000] = 'x';   /* 丙：越界整整一百万字节 */
```

甲几乎必崩；乙崩不崩看运气；丙越界一百万字节，反而是死得最干脆的那个。三段代码在 C11 里全是未定义行为，标准不给任何承诺，可现实里它们的命运却呈现规律：甲的段错误提示 `Segmentation fault`（是否附带 `(core dumped)` 取决于 `ulimit -c` 的设置），乙常在头几条指令就死，丙同样当场段错误。真正"安静"的越界另有其人，它在堆里，第一节结尾揭晓。这一篇把这些命运收拢到一条根因上：崩溃不是 `printf` 报的，不是 C 语言报的，是硬件与操作系统联手判的。篇 16 埋的问题（空指针为什么崩而不是读到 0）在这里拆到底。

一、崩的瞬间：一条三层流水线

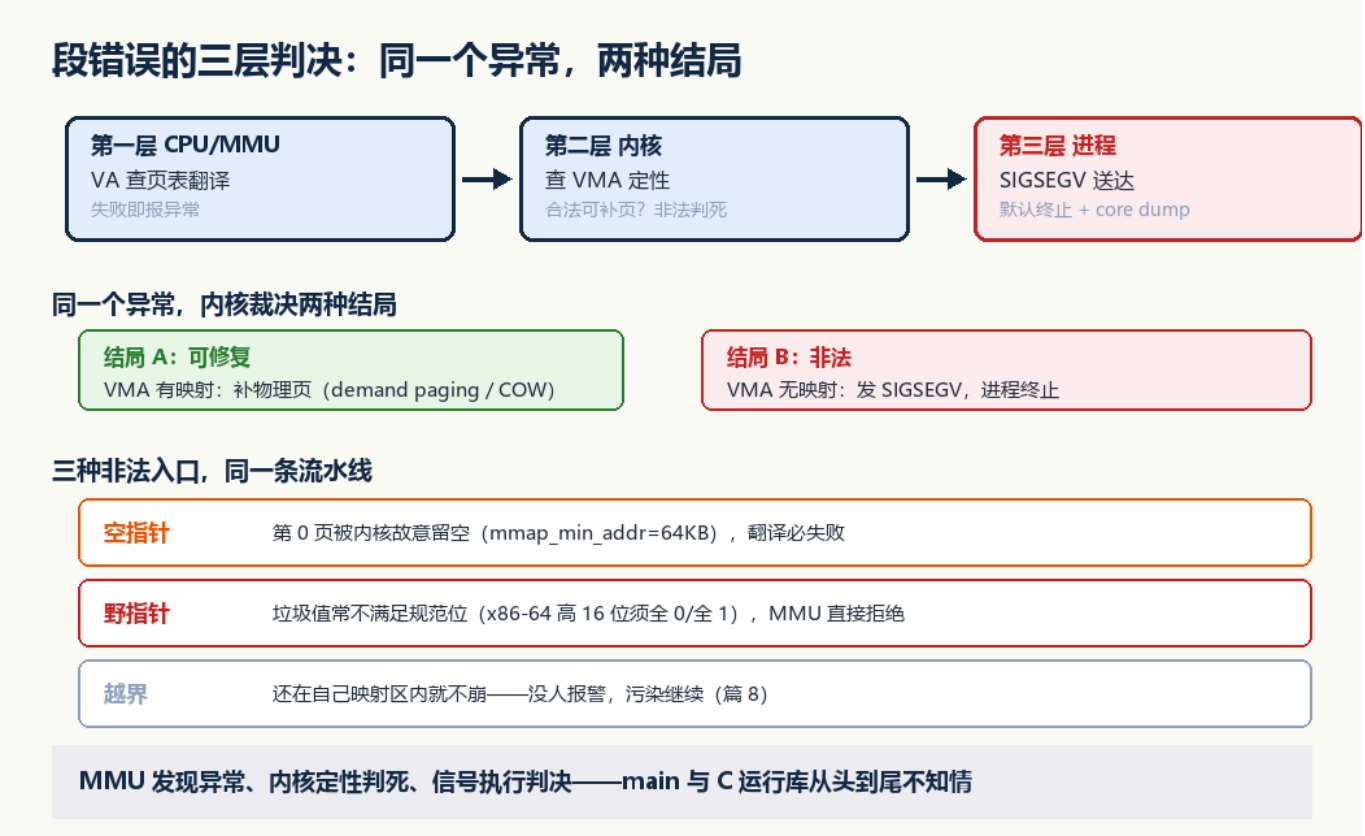
以甲为例，逐步回放 `int a = *p` 的执行现场。

第一层在 CPU 里。`p` 是空指针，值为 0，MMU 拿着虚拟地址 0 查页表：篇 16 讲过，内核故意不映射地址 0 附近那一页，这条翻译查不到任何有效映射，MMU 当场报异常。注意这个异常与篇 16 的缺页异常是同一个异常：demand paging 的补页、COW 的复制，都靠它触发；区别只在内核的裁决——查虚拟地址空间（VMA）发现这段地址没有任何合法映射，无法补页，定性为非法访问。

第二层在内核里。内核确认这是一次进程无法自救的非法访问，于是给进程发送 `SIGSEGV` 信号。信号的名字就是 segment violation 的缩写，段错误的"段"字来自更古老的分段内存时代，名字沿用至今，含义早已变成"页表裁决失败"。

第三层在进程里。进程对 `SIGSEGV` 的默认处置是终止并转储内存（core dumped），你在终端看到的那行 `Segmentation fault (core dumped)` 是 shell 在报告子进程的死因。所以"谁杀

了程序"的完整答案：MMU 发现异常，内核定性判死，信号执行判决，`main` 函数、C 运行库，从头到尾不知情。



图：段错误的三层判决。MMU 报异常与 demand paging 是同一个异常，内核查 VMA 定性：可补页则续命，无映射则发 SIGSEGV。

乙和丙走的是同一条流水线，只是入口不同。乙的 `q` 里是栈上残留的垃圾值，若高位不是规范形态（下一节细说），MMU 直接拒绝，死得极快；碰巧是某个合法旧地址，则读出一个错误的数，不崩但答案错。丙走的是另一条路：`buf` 是 `main` 的栈上局部变量，向高地址写一百万字节，而 `main` 的栈帧紧挨着栈区顶端（上面只剩 argv/环境变量的几 KB），+1MB 早早越过栈顶落进未映射区，大概率当场 SIGSEGV。真正能"安静越界"的现场是另一幅图景：在一大块 `malloc` 出来的堆内存内部越界（只要不出这块的映射边界），页表照常翻译，连异常都不会有，这正是篇 8 的结论：越界不崩比崩更危险，因为没人报警。

二、规范位：野指针死得快的硬件原因

乙的"看运气"里藏着一条硬规则。x86-64 的 64 位地址只实现了 48 位有效位，硬件要求高 16 位必须与第 47 位相同：全 0（用户态范围）或全 1（内核范围），这样的地址叫规范地址（canonical address）。不满足呢？MMU 连页表都不查，直接判定地址非法，抛出保护性异常。

这一条解释了大量"野指针死得快"的现场：未初始化的指针值是栈上旧数据，只要不是恰好的合法地址，大概率不满足规范形态（栈上残留旧地址的高位常带垃圾，形如 `0xdeadbeef0badf00d`），一次解引用立刻异常。顺带纠正一个直觉：`0xdeadbeef` 这类"顺手的"测试值在 64 位下高位全 0，数值上反而是规范地址，它不会被规范位检查拦下，只会死于页表查无映射，结局同样是崩。arm64 同理，48/52 位之外的高位同样是规范位。所以野指针的典型命运不是"悄悄写坏内存"，而是当场被抓——这是硬件在规范位上白送的检查，不是 C 语言的功劳。

三、为什么是"通常"：三个不崩的剧本

标题里的"通常"必须兑现。剧本一，裸机没有这套防护网。Cortex-M 没有 MMU，地址直接上总线：地址 0 处映射的是 Flash 起始区，放着中断向量表，解引用空指针读到的不是异常，是向量表里的数据；若拿它当可写内存去写，破坏的是向量表本身，后果比崩溃隐蔽得多。篇 15 说过裸机上地址是真的，这句话的另一面就是：地址出错也没有裁判，C 的"崩"是带 MMU 系统的奢侈品。

剧本二，优化器可能让崩变得不可预测。解引用空指针是 UB（C11 §6.5.3.2），编译器有权假设它不发生：篇 8 的 UB 折叠在这里同样生效，先解引用后判空的代码，判空可能被整体删除；崩溃位置可能漂移到看似无关的语句。所以"在我机器上没崩"不构成"代码正确"的证据，只构成"还没轮到它"的观察。

剧本三，地址 0 并非永远不可映射。Linux 提供 `mmap` 固定地址映射的能力，历史上 NULL 页可以被映射，攻击者借此把"空指针崩溃"改造成"空指针执行"（NULL dereference exploit）。现代内核默认 `mmap_min_addr` 为 64KB，普通进程再也摸不到第 0 页——空指针"必崩"是内核用一条配置守出来的，不是天条。



图：三个例外剧本与两件工具。裸机无裁判、优化器折叠 UB、第 0 页可被映射；ASan 与 core dump 是把崩用足的手段。

四、崩是恩赐，以及如何接住它

换个视角：SIGSEGV 是操作系统送的最快反馈。程序带着非法地址在内存里游荡，没人报警时它污染的是别人的数据（堆内大块越界的下场）；有人报警时它死在案发现场，core dump 把现场整个封存。工程上要做的不是祈祷不崩，而是把崩用足：

```
gcc -g -fsanitize=address demo.c && ./a.out
# AddressSanitizer: SEGV on unknown address 0x000000000000
#     #0 0x40111a in main demo.c:6
```

AddressSanitizer（`-fsanitize=address`）在每次访存处插检查，把"碰巧不崩"的越界和悬空也抓出来，并直接报出行号；core dump 配 `gdb bt` 能还原崩溃时的完整调用链。再往前一步是预防：指针初始化为 NULL、解引用前判空（注意先解引用后判空等于白判）、`snprintf` 与长度校验拦住篇 49 的缓冲区溢出。最后一条红线：给 `SIGSEGV` 挂自定义处理函数基本无救，硬件异常类的信号，处理函数返回后行为未定义，现场早已不可恢复，接住它的正确方式是让 core dump 落盘。

写在最后

1. 段错误是三层流水线的判决：MMU 翻译失败报异常、内核查 VMA 定性非法、SIGSEGV 终止进程；它与 demand paging 是同一个异常的两种结局，区别只在内核能否补页。
2. 空指针崩的前提是内核故意不映射第 0 页（`mmap_min_addr`），野指针死得快靠规范位（x86-64 高 16 位须全 0/全 1）。"通常"二字背后是硬件与内核的层层设卡。
3. 裸机没有这套防护网：Cortex-M 地址 0 是向量表，空指针读到的不是异常而是数据，写它破坏的更关键。
4. 解引用空指针/野指针/越界全是 UB，优化器折叠让"没崩"不可作为证据；ASan 与 core dump 是把崩用足的两件工具。
5. `SIGSEGV` 处理函数返回后行为未定义，现场不可恢复；防御的位置在写代码时：判空、限长、初始化。

内存系列刚开头，下一站是每个 C 程序员都该亲眼看过一次的现场：《函数调用时栈里发生了什么：栈帧与调用约定》，把一次函数调用的入栈、传参、返回拆到寄存器和内存条上。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。